

NanoVDB `ReadAccessor` Benchmark — OLD vs NEW, `Acc=<0,1,2>` vs `Acc=<0>`, CPU vs GPU

Benchmarks comparing the `NANOVDB_USE_OLD_ACCESSOR` caching behaviour (ON vs OFF) and `ReadAccessor<0,1,2>` (full 3-level cache, default) vs `ReadAccessor<0>` (leaf-only cache) across access patterns, on CPU (single- and multi-threaded) and GPU. Results are collected from five machines: a **Razer laptop**, a **Dell laptop**, a **desktop**, a **workstation**, and a **MacBook Air M3**. The full accessor-type comparison (both `<0,1,2>` and `<0>`, corrected leaf-sampling methodology) was run on the Dell laptop, Desktop, Workstation, Razer Laptop, and MacBook Air M3.

⚠ Methodology note (Razer Laptop section only). That section sampled a cube inside a **fog-volume sphere**. That interior is stored as **active tiles at upper internal nodes**, so every access resolved at level 2 and **never reached a leaf** — which makes the leaf-only `ReadAccessor<0>` look artificially bad and does not measure genuine leaf traversal. The Dell laptop, MacBook Air M3, Workstation, and Desktop sections use a **corrected** benchmark whose coordinates are harvested from the grid's **active leaf voxels** (a narrow-band level set sphere); 100 % of accesses resolve at a leaf node. Consequently the **absolute ns/access in those sections are ~10× larger** than the pre-correction Razer Laptop section — that difference is the methodology (real multi-level leaf traversal vs. re-hitting a small set of cached upper-node tiles), **not** the hardware. Compare OLD-vs-NEW and `<0,1,2>` -vs- `<0>` only **within** a section, never across the correction boundary.

What is being measured

The `ReadAccessor` caches the tree nodes visited on the previous access so a spatially nearby access can skip the traversal from the root.

- **OLD** (`NANOVDB_USE_OLD_ACCESSOR` defined): the `get()` method uses

`if constexpr + else` chaining. For `getValue` (operation `LEVEL=0`), the `if constexpr(LEVEL<=0)` branch is taken and the `else` discards all higher-level cache checks — so the 3-level accessor falls straight to a **root traversal** on any leaf-cache miss, effectively behaving like a leaf-only accessor.

- **NEW** (`NANOVDB_NO_OLD_ACCESSOR` defined): the `else` is removed, so all

three cache-level checks execute unconditionally. A leaf-cache miss can still hit the **level-1 / level-2** cache instead of restarting at the root.

- `ReadAccessor<0,1,2>` (`DefaultReadAccessor`): maintains leaf, lower-internal,

and upper-internal node caches. With NEW, the full 3-level check is active. With OLD, the extra levels are stored but never checked on a read — making it functionally equivalent to a larger `ReadAccessor<0>` struct.

- `ReadAccessor<0>` : caches only the leaf node. On a leaf miss it always goes

straight to root. Unaffected by the OLD/NEW flag (no level-1/2 to check).

Methodology

What the numbers measure

Every reported figure is the **wall-clock time of a single random-access read** into the grid, i.e. one `accessor.getValue(ijk)` call, expressed in **nanoseconds per lookup** (`ns/lookup`). It is a pure **read-path** micro-benchmark: no values are written, no math is done on the result beyond a running sum (kept `volatile` / written to an output array so the compiler cannot elide the work).

- **CPU-1T** — one thread walks the coordinate list serially with a single

accessor. This is a **latency** number: the cost of one dependent lookup.

- **CPU-MT** — the same total work split across all hardware threads

(`nanovdb::util::forEach` → `tbb::parallel_for`), each grain using its own accessor. This is a **throughput** number: total time ÷ total lookups with all cores busy.

- **GPU** — the coordinate list is uploaded to the device; each thread processes

a 32-coord chunk with its own accessor. Timed with CUDA events (kernel time only, excluding the one-time grid upload). Also a **throughput** number.

The only thing that differs between the OLD and NEW columns is the accessor's cache-fallback logic (a compile-time switch); the data, coordinates, and harness are identical.

The test data

A **fog-volume sphere** built with `nanovdb::tools::createFogVolumeSphere<float>(radius=500, voxelSize=1.0, halfWidth=3.0)`. Its measured properties:

Property	Value
Value type	<code>float</code>
Index bounding box	<code>[-500, -500, -500] ... [500, 500, 500]</code> (1001 ³ voxels)
Dense voxel count of bbox	1,003,003,001 (~1.00 B)
Active voxels	523,592,077 (~523.6 M)
Fill ratio (active / dense bbox)	52.2 %
Upper internal nodes (level 2, 32 ³)	8
Lower internal nodes (level 1, 16 ³)	272
Leaf nodes (level 0, 8 ³)	82,608
Grid size in memory	188,484,640 bytes (~180 MB)

So it is a **fully populated solid ball** — the interior is all active voxels (the 52 % fill ratio is just the volume of a sphere inside its cube, $\pi/6 \approx 0.524$). It is **sparse** in the VDB sense (only the ball is stored, not the empty corners), but the region the benchmark actually samples is **100 % active**: all access patterns are confined to a cube well inside radius 500, so every lookup lands on a real leaf and traverses the tree for real — we are **not** measuring empty-space shortcuts, we are measuring genuine node traversal and cache reuse.

How much work, how it is averaged

Point patterns (Sequential / LeafJump / NodeJump / Random)	1,048,576 lookups each
Stencil	262,144 centres × 27 neighbours = 7,077,888 lookups
Sampled region	cube of half-extent 256 (points) / <code>[0, 64)³</code> dense cube (stencil), both fully inside the ball
Repetition	each configuration run 7 times; the median is reported (robust to OS jitter / turbo ramp)
Warm-up	GPU does one untimed warm-up launch before the 7 timed trials

Hardware / build

Five machines were benchmarked; results for each are in separate sections below.

Razer Laptop		Dell laptop	Desktop	Workstation
CPU	Intel Core i7-9750H — 6 cores / 12 threads; TBB <code>parallel_for</code> , 4096-coord grains	32 HW threads; TBB <code>parallel_for</code> , 4096-coord grains	AMD Ryzen 9 9950X — 16 cores / 32 threads; same TBB settings	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64 threads; same TBB settings
GPU	NVIDIA Quadro RTX 5000 with Max-Q Design (SM 7.5); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX 5000 Ada Generation Laptop GPU (SM 8.9); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (SM 12.0); same CUDA settings	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); same CUDA settings
Build	Release (<code>-O3 -DNDEBUG</code>), C++17 / CUDA 17	same	same	same
OLD vs NEW	selected at compile time per target (<code>-DNANOVDB_USE_OLD_ACCESSOR</code> vs <code>-DNANOVDB_NO_OLD_ACCESSOR</code>)	same	same	same

Access patterns

Pattern	Stride / shape	Cache behaviour
Sequential	1	leaf (level-0) cache always hot
LeafJump	8 (= leaf dim)	leaf cache cold; level-1 warm (NEW only)
NodeJump	128 (= lower-node dim)	leaf + level-1 cold; level-2 warm (NEW only)
Random	uniform in cube	all caches cold — both fall to root
Stencil	3×3×3 = 27 neighbours per centre, dense sweep	mostly same-leaf hits; boundary neighbours spill to level-1 (NEW rescues)

Results — MacBook Air M3 (Apple M3, 8-core CPU; corrected benchmark, CPU only)

This section uses the corrected leaf-sampling methodology (see the ⚠ note above). Coordinates are harvested from the active leaf voxels of a narrow-band level set sphere: `nanovdb::tools::createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Upper internal nodes (level 2)	8
Access resolving at leaf (verified)	100 % (was 0 % pre-correction)
CPU	Apple M3, 8 HW threads; TBB <code>parallel_for</code> , 4096-coord grains
GPU	none (no CUDA on Apple silicon) — CPU only
Build	Release (<code>-O3 -DNDEBUG</code>), C++17; each printed figure is the median of 7 trials

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>	OLD → NEW <0,1,2>
Sequential	20.98	20.35	19.79	20.04	1.06×
LeafJump	135.47	108.52	53.17	104.53	2.55×
NodeJump	123.07	109.29	90.77	107.62	1.36×
Random	218.39	176.14	185.11	165.49	1.18×
Stencil (ns/lookup)	52.06	47.97	40.44	47.43	1.29×

CPU 8-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>	OLD → NEW <0,1,2>
Sequential	3.98	3.76	3.82	3.73	1.04×
LeafJump	26.13	21.02	11.63	20.70	2.25×
NodeJump	26.83	20.96	18.36	20.69	1.46×
Random	34.67	30.94	37.92	28.90	0.91×
Stencil (ns/lookup)	11.51	11.36	8.79	10.05	1.31×

Stencil detail — ns per whole 27-neighbour stencil

Mode	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	1405.57	1295.28	1091.88	1280.73
CPU 8 threads	310.72	306.74	237.44	271.33

Takeaways — MacBook Air M3 (corrected benchmark)

- ReadAccessor<0> is unchanged by OLD → NEW (within ~1–6 % on every pattern and thread count, i.e. noise). This is the clean control the corrected sampling makes possible — the OLD/NEW flag only touches the multi-level cache chaining in <0,1,2> .
- <0,1,2> NEW wins exactly where multi-level caching applies: **LeafJump 2.55× (1T) / 2.25× (8T)** — the headline — plus NodeJump 1.36–1.46× and Stencil 1.29–1.31×. Sequential is a tie (access stays within one leaf, which both variants cache).
- Random is the one place <0,1,2> does not help (0.91× on 8T; the 1.18× on 1T is inflated because random picks among the clustered narrow-band voxels retain some upper-node locality). For pure scatter, <0> remains the better choice — it avoids the extra cache-level checks.

Results — Desktop (AMD Ryzen 9 9950X / RTX PRO 6000 Blackwell, SM 12.0) — corrected benchmark

This section uses the **corrected leaf-sampling benchmark** (same methodology as the MacBook Air M3 and Workstation sections). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere (`createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`); every lookup resolves at a leaf node.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Access resolving at leaf	100 %
CPU	AMD Ryzen 9 9950X — 16 cores / 32 threads; TBB <code>parallel_for</code> , 4096-coord grains
GPU	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (SM 12.0); 32-coord chunks/thread, 128 threads/block
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	1.52	0.91	1.47	0.85
LeafJump	7.25	5.52	4.39	5.58
NodeJump	2.96	2.97	4.01	3.00
Random	21.81	21.45	23.13	20.21
Stencil (ns/lookup)	1.112	1.113	1.141	1.234

CPU 32-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.17	0.17	0.19	0.15
LeafJump	0.46	0.50	0.47	0.51
NodeJump	0.28	0.40	0.37	0.31
Random	1.21	1.12	1.44	1.28
Stencil (ns/lookup)	0.107	0.091	0.096	0.093

GPU — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.027	0.026	0.026	0.027
LeafJump	0.074	0.073	0.050	0.073
NodeJump	0.087	0.087	0.066	0.087
Random	0.096	0.096	0.121	0.096
Stencil (ns/lookup)	0.0691	0.0689	0.0630	0.0689

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	1.03×	1.07×	0.89×	1.13×	1.04×	0.96×
LeafJump	1.65×	0.99×	0.98×	0.98×	1.48×	1.00×
NodeJump	0.74×	0.99×	0.76×	1.29×	1.32×	1.00×
Random	0.94×	1.06×	0.84×	0.88×	0.79×	1.00×
Stencil	0.97×	0.90×	1.11×	0.98×	1.10×	1.00×

Key: **ReadAccessor<0>** is essentially unchanged by OLD → NEW — confirming the fix only affects the 3-level accessor. NEW <0,1,2> wins strongly for LeafJump (CPU-1T 1.65×, GPU 1.48×). NodeJump shows a notable CPU-1T regression (0.74×) because with the corrected methodology (one real leaf per lower-internal node), the level-1 cache always misses and the extra checks add latency; the GPU benefits from the level-2 hit path (1.32×) despite the same pattern.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	30.03	30.04	30.81	33.33
CPU 32 threads	2.88	2.45	2.60	2.50
GPU	1.8660	1.8616	1.7002	1.8613

The GPU stencil improves 10 % with NEW <0,1,2> (1.7002 vs 1.8660 ns/stencil) — in contrast to the Workstation (RTX 6000 Ada, SM 8.9) where it regressed 14 %. On the RTX PRO 6000 Blackwell (SM 12.0) the extra cache-level checks are absorbed and the level-1/2 cache hits across stencil neighbourhood boundaries produce a net gain. ReadAccessor<0> (GPU) is unchanged (1.00×), as expected — it has no level-1/2 logic for the flag to affect.

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0> (0.85 ns 1T)	Leaf cache stays hot; level-1/2 checks add marginal cost
LeafJump	<0,1,2> (4.39 ns)	Level-1 cache rescues leaf misses; 1.65× vs <0>
NodeJump	<0> (3.00 ns)	CPU: level-1 always misses, extra checks add latency; GPU: <0,1,2> wins
Random	<0> (20.21 ns)	All levels miss; extra checks are pure cost
Stencil (CPU)	<0,1,2> (1.112 ns/lkp OLD, 0.096 ns/lkp MT)	MT edge (1.11×); <0> regresses on 1T
Stencil (GPU)	<0,1,2> (0.0630 ns/lkp)	Blackwell benefits: 10 % faster than OLD or <0>

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-32T	GPU	GPU vs CPU-32T
Sequential (<0>)	0.85	0.15	0.026	5.8×
LeafJump (<0,1,2>)	4.39	0.47	0.050	9.4×
Random (<0>)	20.21	1.28	0.096	13.3×
Stencil (<0,1,2>)	1.141	0.096	0.0630	1.5×

Results — Dell laptop (RTX 5000 Ada Generation Laptop GPU, SM 8.9) — corrected benchmark

This section now uses the **corrected leaf-sampling benchmark** (same methodology as the MacBook Air M3, Desktop, and Workstation sections). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere (`createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`); every lookup resolves at a leaf node. Both `ReadAccessor<0,1,2>` and `ReadAccessor<0>` are benchmarked under OLD and NEW.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Upper internal nodes (level 2)	8
Access resolving at leaf	100 %
CPU	32 HW threads; TBB <code>parallel_for</code> , 4096-coord grains
GPU	NVIDIA RTX 5000 Ada Generation Laptop GPU (SM 8.9); 32-coord chunks/thread, 128 threads/block
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	1.63	1.34	1.49	1.34
LeafJump	10.23	9.62	6.48	9.63
NodeJump	5.06	5.07	4.35	5.02
Random	31.12	29.75	32.60	29.90
Stencil (ns/lookup)	2.013	1.945	2.469	2.153

CPU 32-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.43	0.40	0.45	0.43
LeafJump	0.85	0.86	0.69	0.88
NodeJump	0.68	0.66	0.63	0.65
Random	1.61	1.62	1.96	1.66
Stencil (ns/lookup)	0.188	0.200	0.171	0.179

GPU — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.065	0.066	0.067	0.066
LeafJump	0.112	0.112	0.097	0.112
NodeJump	0.150	0.149	0.110	0.149
Random	0.221	0.222	0.234	0.222
Stencil (ns/lookup)	0.0845	0.0842	0.0958	0.0845

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	1.09×	1.00×	0.96×	0.93×	0.97×	1.00×
LeafJump	1.58×	1.00×	1.23×	0.98×	1.15×	1.00×
NodeJump	1.16×	1.01×	1.08×	1.02×	1.36×	1.00×
Random	0.95×	1.00×	0.82×	0.98×	0.94×	1.00×
Stencil	0.82×	0.90×	1.10×	1.12×	0.88×	1.00×

Key: **ReadAccessor<0>** is essentially unchanged by OLD → NEW — confirming the fix only touches the 3-level accessor (the ~1.10× on <0> stencil is laptop thermal/turbo noise on the serial run, not a real effect). NEW <0,1,2> wins strongly for **LeafJump (CPU-1T 1.58×, CPU-MT 1.23×, GPU 1.15×)** and **NodeJump on GPU (1.36×)**. This machine (SM 8.9 Ada) reproduces the Workstation's GPU-stencil regression (0.88×): with the corrected active-voxel stencil the taps stay in-band, the leaf cache is already hot, and the extra level-1/2 checks add overhead with no benefit.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	54.36	52.52	66.65	58.12
CPU 32 threads	5.07	5.40	4.62	4.82
GPU	2.2813	2.2728	2.5867	2.2813

On CPU the multi-threaded stencil favours NEW <0,1,2> (4.62 vs 5.07 ns/stencil, 1.10×), but the single-threaded stencil regresses (66.65 vs 54.36) — the 1T serial runs are the noisiest on this thermally-constrained laptop. The GPU (SM 8.9 Ada) regresses 12 % with NEW <0,1,2>, matching the Workstation's SM 8.9 result; **ReadAccessor<0>** is neutral on GPU (1.00×) as expected.

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0> (1.34 ns 1T)	Leaf cache stays hot; level-1/2 checks add marginal cost
LeafJump	<0,1,2> (6.48 ns)	Level-1 cache rescues leaf misses; 1.58× vs <0>
NodeJump	<0,1,2> (4.35 ns; GPU 0.110 vs 0.149)	Level-1/2 cache pays off on both CPU and GPU
Random	<0> (29.90 ns)	All levels miss; extra checks are pure cost
Stencil (CPU)	<0,1,2> (0.171 ns/lkp MT)	MT edge (1.10×); 1T noisy
Stencil (GPU)	<0> (0.0845 ns/lkp)	NEW <0,1,2> regresses 12 %; leaf cache already hot

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-32T	GPU	GPU vs CPU-32T
Sequential (<0>)	1.34	0.43	0.066	6.5×
LeafJump (<0,1,2>)	6.48	0.69	0.097	7.1×
Random (<0>)	29.90	1.66	0.222	7.5×
Stencil GPU (<0>)	—	—	0.0845	—

Results — Workstation (AMD Ryzen Threadripper PRO 7975WX / RTX 6000 Ada, SM 8.9) — corrected benchmark

This section uses the **corrected leaf-sampling benchmark** (same methodology as the MacBook Air M3 section). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere (`createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`); every lookup resolves at a leaf node.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Access resolving at leaf	100 %
CPU	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64 threads; TBB <code>parallel_for</code> , 4096-coord grains
GPU	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); 32-coord chunks/thread, 128 threads/block
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	1.59	1.29	1.26	1.10
LeafJump	8.06	6.44	4.41	6.52
NodeJump	5.30	4.32	4.67	4.47
Random	23.09	19.04	27.28	19.00
Stencil (ns/lookup)	1.857	1.561	1.761	1.716

CPU 64-threaded — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.22	0.18	0.22	0.18
LeafJump	0.36	0.37	0.31	0.32
NodeJump	0.29	0.27	0.34	0.29
Random	0.74	0.68	0.81	0.67
Stencil (ns/lookup)	0.066	0.077	0.065	0.073

GPU — ns per access (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.027	0.026	0.026	0.026
LeafJump	0.063	0.063	0.045	0.063
NodeJump	0.086	0.086	0.058	0.083
Random	0.133	0.132	0.147	0.132
Stencil (ns/lookup)	0.0587	0.0586	0.0669	0.0583

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	1.26×	1.17×	1.00×	1.00×	1.04×	1.00×
LeafJump	1.83×	0.99×	1.16×	1.16×	1.40×	1.00×
NodeJump	1.13×	0.97×	0.85×	0.93×	1.48×	1.04×
Random	0.85×	1.00×	0.91×	1.01×	0.90×	1.00×
Stencil	1.05×	0.91×	1.02×	1.05×	0.88×	1.01×

Key: **ReadAccessor<0>** is essentially unchanged by OLD → NEW — confirming that the fix only affects the 3-level accessor. NEW <0,1,2> wins strongly for LeafJump (CPU 1T 1.83×, GPU 1.40×) and NodeJump on GPU (1.48×). Stencil shows only a small CPU win and a slight GPU regression — explained below.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	50.14	42.14	47.53	46.34
CPU 64 threads	1.79	2.08	1.75	1.97
GPU	1.5859	1.5828	1.8057	1.5742

The GPU stencil **regresses 14 % with NEW <0,1,2>** (1.8057 vs 1.5859 ns/stencil). With the corrected methodology the stencil centres are active interior-band voxels whose full 27-neighbour neighbourhood is also active — meaning all taps stay within the narrow leaf band, the level-0 (leaf) cache is already hot, and the extra level-1/2 checks in NEW add overhead with no benefit. This contrasts with the fog-sphere stencil (old methodology) where sampling a dense cube caused cross-leaf spills that the level-1 cache caught.

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0> (1.10 ns 1T)	Leaf cache stays hot; level-1/2 checks add marginal cost
LeafJump	<0,1,2> (4.41 ns)	Level-1 cache rescues leaf misses; 1.83× vs <0>
NodeJump	<0,1,2> (4.67 ns) or <0> (4.47 ns)	Level-2 gives slight edge on GPU; effectively tied on CPU
Random	<0> (19.00 ns)	All levels miss; extra checks are pure cost
Stencil (CPU)	<0,1,2> (1.761 ns/lkp 1T)	Small edge; taps stay in-leaf so benefit is modest
Stencil (GPU)	<0> (0.0583 ns/lkp)	NEW <0,1,2> regresses; leaf cache already hot

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-64T	GPU	GPU vs CPU-64T
Sequential (<0>)	1.10	0.18	0.026	6.9×
LeafJump (<0,1,2>)	4.41	0.31	0.045	6.9×
Random (<0>)	19.00	0.67	0.132	5.1×
Stencil CPU (<0,1,2>)	1.761	0.065	—	—
Stencil GPU (<0>)	—	—	0.0583	~1.1× vs CPU-64T

Results — Razer Laptop (Intel Core i7-9750H / Quadro RTX 5000 Max-Q, SM 7.5)

Both `ReadAccessor<0,1,2>` and `ReadAccessor<0>` were benchmarked under both OLD and NEW compile-time modes. The CPU is a Turing-era mobile part with 6 physical cores / 12 HW threads; the GPU is an NVIDIA Quadro RTX 5000 with Max-Q Design (Turing, SM 7.5).

CPU single-threaded — ns per lookup (latency)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	5.17	5.44	3.21	5.26
LeafJump	5.38	5.61	3.67	5.51
NodeJump	5.31	5.32	4.72	5.32
Random	11.42	11.94	16.13	11.53
Stencil (ns/lookup)	5.200	5.700	2.959	5.654

CPU 12-threaded — ns per lookup (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	1.50	1.36	0.68	1.22
LeafJump	1.25	1.46	0.72	1.30
NodeJump	1.47	1.80	1.11	1.24
Random	1.70	1.66	2.42	1.81
Stencil (ns/lookup)	1.553	1.462	0.738	1.314

GPU — ns per lookup (throughput)

Pattern	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
Sequential	0.302	0.309	0.344	0.283
LeafJump	0.293	0.302	0.336	0.277
NodeJump	0.261	0.268	0.275	0.248
Random	0.363	0.378	0.388	0.345
Stencil (ns/lookup)	0.1999	0.2098	0.0849	0.1858

OLD → NEW speedup by accessor type

Pattern	CPU-1T <0,1,2>	CPU-1T <0>	CPU-MT <0,1,2>	CPU-MT <0>	GPU <0,1,2>	GPU <0>
Sequential	1.61×	1.03×	2.21×	1.11×	0.88×	1.09×
LeafJump	1.47×	1.02×	1.74×	1.12×	0.87×	1.09×
NodeJump	1.12×	1.00×	1.32×	1.45×	0.95×	1.08×
Random	0.71×	1.04×	0.70×	0.92×	0.94×	1.10×
Stencil	1.76×	1.01×	2.10×	1.11×	2.35×	1.13×

Key: **ReadAccessor<0>** is essentially unchanged by OLD → NEW on CPU, in line with all other machines. On the GPU (SM 7.5 Turing), <0> shows a small but consistent ~1.09–1.13× improvement — an outlier not seen on newer architectures, possibly noise or a compiler code-gen difference on this older target.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD <0,1,2>	OLD <0>	NEW <0,1,2>	NEW <0>
CPU 1 thread	140.39	153.90	79.89	152.65
CPU 12 threads	41.94	39.48	19.92	35.48
GPU	5.3960	5.6641	2.2926	5.0156

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	<0,1,2> (3.21 ns)	Level-1 cache stays warm between leaf-boundary crossings
LeafJump	<0,1,2> (3.67 ns)	Same — level-1 rescues leaf misses
NodeJump	<0,1,2> (4.72 ns)	Level-2 cache pays off; slightly faster than <0> (5.32 ns)
Random	<0> (11.53 ns)	All levels miss; extra checks are pure cost
Stencil	<0,1,2> (2.959 ns/lkp)	Boundary neighbours caught by level-1; ~1.91× over <0>

Unlike the Workstation (where NodeJump NEW <0,1,2> regressed 2× vs OLD), here the level-2 cache benefit slightly outweighs the extra check overhead, so <0,1,2> remains the right choice for NodeJump as well.

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-12T	GPU	GPU vs CPU-12T
Sequential (<0,1,2>)	3.21	0.68	0.283	2.4×
Random (<0>)	11.53	1.81	0.345	5.2×
Stencil (<0,1,2>)	2.959	0.738	0.0849	8.7×

Cross-machine comparison (Razer Laptop vs Dell laptop vs Desktop vs Workstation)

*Scope: the Dell laptop, Desktop, Workstation, MacBook Air M3, and Razer Laptop all use both accessor types. The Dell laptop, Desktop, Workstation, and MacBook Air M3 use the **corrected** leaf-sampling methodology; only the Razer Laptop section is still on the pre-correction fog-sphere methodology, whose absolute ns/access values are not directly comparable. Compare OLD → NEW ratios and <0,1,2> -vs- <0> ratios within each section.*

1. LeafJump is the biggest win with corrected methodology

With coordinates drawn from actual active leaf voxels, LeafJump (one voxel per leaf in storage order) exercises a realistic cache pattern: the level-0 cache always misses, but adjacent leaves share the same lower-internal node — exactly the case NEW <0,1,2> rescues.

Platform	LeafJump CPU-1T OLD → NEW <0,1,2>	LeafJump GPU OLD → NEW <0,1,2>
Desktop (corrected, RTX PRO 6000 Blackwell)	1.65×	1.48×
Workstation (corrected, RTX 6000 Ada)	1.83×	1.40×
Dell laptop (corrected, RTX 5000 Ada)	1.58×	1.15×
Razer Laptop (pre-correction, RTX 5000 Turing)	1.47×	0.87×

The corrected results (Desktop, Workstation, Dell laptop) represent the true benefit more accurately than the pre-correction figures. The Razer's GPU regression was already flagged as a SM 7.5 anomaly.

2. GPU NodeJump wins with corrected methodology

Platform	NodeJump GPU OLD → NEW <0,1,2>
Desktop (corrected, RTX PRO 6000 Blackwell)	1.32×
Workstation (corrected, RTX 6000 Ada)	1.48×
Dell laptop (corrected, RTX 5000 Ada)	1.36×
Razer Laptop (pre-correction)	0.95×

With real level-2 cache locality (one voxel per lower-internal node from the actual grid topology, staying within the same upper node), the level-2 cache hit path in NEW is exercised and pays off on GPU. This was not visible in the synthetic NodeJump (fixed stride=128) used in pre-correction runs.

3. Stencil GPU: corrected methodology result is architecture-dependent

Machine	GPU Stencil OLD → NEW <0,1,2>	Methodology
Desktop (corrected, SM 12.0 Blackwell)	1.10× (improvement)	active-voxel, 27-tap-active filter
Workstation (corrected, SM 8.9 Ada)	0.88× (regression)	active-voxel, 27-tap-active filter
Dell laptop (corrected, SM 8.9 Ada)	0.88× (regression)	active-voxel, 27-tap-active filter
Razer Laptop (pre-correction, SM 7.5 Turing)	2.35×	fog sphere, dense cube

With the corrected stencil (centres where all 27 neighbours are active leaf voxels), taps stay within the narrow band and the level-0 leaf cache is already hot. On **SM 8.9 (Ada)** — both the Workstation (RTX 6000 Ada) and the Dell laptop (RTX 5000 Ada) — the extra level-1/2 checks in NEW add overhead with no net cache benefit → **12–14 % regression** (a clean cross-machine reproduction). On SM 12.0 (Blackwell), the architectural improvements absorb the extra checks and the level-1/2 hits at neighbourhood boundaries produce a 10 % gain. ReadAccessor<0> is neutral on GPU (1.00×) in all cases. The large pre-correction ~2× GPU wins were real for the dense fog-sphere use case but do not represent narrow-band workloads.

4. CPU MT pattern: modest wins for LeafJump; mixed elsewhere

Pattern	Desktop (corrected, 32T) CPU-MT <0,1,2>	Workstation (corrected, 64T) CPU-MT <0,1,2>	Dell laptop (corrected, 32T) CPU-MT <0,1,2>
Sequential	0.89×	1.00×	0.96×
LeafJump	0.98×	1.16×	1.23×
NodeJump	0.76×	0.85×	1.08×
Random	0.84×	0.91×	0.82×
Stencil	1.11×	1.02×	1.10×

With the corrected methodology the MT picture shows modest or neutral changes. The Desktop (32-thread 9950X) and Dell laptop both show a small stencil MT gain (1.11× / 1.10×) and the Dell laptop's best MT LeafJump win (1.23×), while the Workstation (64-thread Threadripper) is near-neutral. The large pre-correction MT stencil wins (~1.57–1.67×) do not appear in any corrected run, consistent with the stencil GPU finding above.

5. ReadAccessor<0> unaffected by OLD → NEW on all platforms

Confirmed: <0> OLD → NEW ratios remain 0.97–1.05× across all patterns, platforms, and methodologies. The fix is entirely in the 3-level <0,1,2> accessor.

6. Random always regresses for <0,1,2>; <0> is neutral

For <0,1,2> : all caches miss → 0.85× on CPU-1T, 0.90× on GPU (corrected run). For <0> : no extra checks → 1.00× on all platforms. Hardware-independent.

Summary and bottom line

The NEW fix is unambiguously beneficial for ReadAccessor<0,1,2> on stencil and coherent point patterns. The gains are 1.5–2.3× and portable across CPU and GPU. ReadAccessor<0> (leaf-only) sees no benefit from the fix and no regression — it is effectively unchanged.

For production use with the NEW accessor, the choice of accessor type matters:

Use case	Recommended accessor
Stencil / convolution / dense sweep	ReadAccessor<0,1,2> (NEW gains 2×)
Sequential / LeafJump access	ReadAccessor<0,1,2> (NEW gains 1.4–2.3×)
NodeJump / random scatter	ReadAccessor<0> (avoids regression, same or faster)
Unknown / mixed	ReadAccessor<0,1,2> — wins on the common workloads

Per-machine takeaways

- LeafJump is the primary win with the corrected methodology (active-voxel patterns): 1.65×/1.48× (Desktop), 1.83×/1.40× (Workstation), and 1.58×/1.15× (Dell laptop) on CPU-1T/GPU for NEW <0,1,2> . This is the pattern that directly exercises the level-1 cache rescue: leaf cache misses but the lower internal node is still cached.
 - Stencil GPU results are architecture-dependent with the corrected methodology.
- On SM 12.0 (Blackwell): NEW <0,1,2> improves 10 %. On SM 8.9 (Ada): it regresses 12–14 %, now confirmed on two independent SM 8.9 machines (Workstation RTX 6000 Ada and Dell laptop RTX 5000 Ada — 0.88× on both). Pre-correction (fog sphere, dense cube): <0,1,2> gained ~2× on GPU across all machines. ReadAccessor<0> is neutral in all cases.
- GPU NodeJump wins with corrected patterns: **1.32× (Desktop), 1.48× (Workstation), 1.36× (Dell laptop)**; with one representative voxel per actual lower-internal node, the level-2 cache hit path in NEW pays off. Not captured by the synthetic stride=128 pattern in pre-correction runs.
 - NodeJump CPU-1T regresses for <0,1,2> on the Desktop (0.74×): level-1 always misses with the real topology pattern; the extra cache checks add latency with no benefit. The GPU still wins (1.32×) because the level-2 hit at the upper-node boundary outweighs check overhead.
 - Random always regresses for <0,1,2> (0.79–0.94× across all platforms): all cache levels miss, extra checks are pure overhead. <0> is neutral (1.00×). Hardware-independent.
 - ReadAccessor<0> is unaffected by OLD → NEW on all platforms, patterns, and thread counts.
- Its code path (leaf check → root) has no level-1/2 checks to change.
- CPU multi-thread scaling: On the 32-thread 9950X (Desktop), LeafJump achieves ~15× (MT vs 1T), Random ~16×, Sequential ~6×. On the 64-thread Threadripper PRO (Workstation), LeafJump achieves 13–16×, Random 29–31×, Sequential 7×.

Building & running

```
# Configure with the benchmark (and CUDA, for the GPU variant) enabled
cmake -S . -B build -DNANOVDDB_BUILD_BENCHMARK=ON -DNANOVDDB_USE_CUDA=ON

# Build all four executables
cmake --build build --target \
    bench_accessor_old bench_accessor_new \
    bench_accessor_cuda_old bench_accessor_cuda_new -j

# Run – each binary prints results for both ReadAccessor<0,1,2> and ReadAccessor<0>
cd build/nanovdb/nanovdb/benchmark
./bench_accessor_old      # CPU, OLD accessor, both <0,1,2> and <0>
./bench_accessor_new      # CPU, NEW accessor, both <0,1,2> and <0>
./bench_accessor_cuda_old  # GPU, OLD accessor, both <0,1,2> and <0>
./bench_accessor_cuda_new  # GPU, NEW accessor, both <0,1,2> and <0>
```

The OLD vs NEW behaviour is selected at compile time per target (`-DNANOVDDB_USE_OLD_ACCESSOR` vs `-DNANOVDDB_NO_OLD_ACCESSOR`). Both accessor types (`ReadAccessor<0,1,2>` and `ReadAccessor<0>`) are run within each binary.

Files

File	Purpose
<code>BenchPatterns.h</code>	Shared access-pattern generation (identical coords for CPU & GPU)
<code>BenchAccessor.cc</code>	CPU benchmark (single- and multi-threaded)
<code>BenchAccessorCuda.cu</code>	GPU (CUDA) benchmark
<code>CMakeLists.txt</code>	Builds the OLD/NEW × CPU/GPU targets